

Revenue is up. That's the problem.

Farrah pinged you before 9am: this month's revenue number looks **too good**. Not wrong in the obvious way — the query runs fine, the number looks plausible, the board deck is due Friday. But something about it bothers her, and now it bothers you too. Time to learn what happens when Solace's tables don't just disagree — they multiply.

UNLOCKS: JOIN FLUENCY

1

2

3

4

5

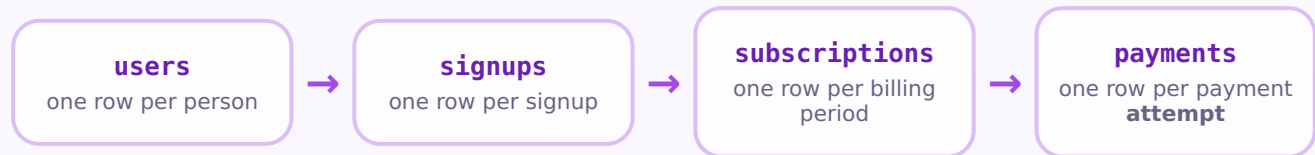
6

7

8

The four tables of Solace

Four tables, connected like stops on a line. Most days that's fine. Today, one connection has more trains running through it than anyone accounted for.



The last stop is the trap: a subscription can have **more than one** payment row this month — a failed retry, then a success, occasionally a refund. Join straight into it without thinking, and every subscription with 2 payment rows quietly counts itself twice.

The JOIN that duplicated revenue

LOCKED — you can't move to the next mission until you find the bug.

Here's the query behind this month's revenue number:

```
SELECT COUNT(s.id) AS active_subs, SUM(p.amount) AS revenue
FROM subscriptions s
JOIN payments p ON p.subscription_id = s.id
WHERE s.status = 'active'
AND p.charged_at >= '2026-06-01';
```

What's wrong with this JOIN? Write it before you turn the page.

No peeking — the bug's explained on the next page, not this one.

LOCK PICKED

Why the JOIN duplicated revenue

Payments is *one-to-many* against subscriptions — a subscription with a failed retry and a successful charge this month has two payment rows. The JOIN duplicates the subscription for every matching payment row, so COUNT(s.id) counts it twice and SUM(p.amount) adds both charges, even the failed one if the status filter is missing. This is called a **fan-out**: the "one" side of the join gets multiplied by the "many" side.

Three ways to join subscriptions to payments

No single right answer — each trades simplicity for safety differently.

Approach	How it works	Trade-off
1. Pre-aggregate payments first	Build a CTE that SUMs payments per subscription_id <i>before</i> joining, so the join is 1-to-1.	Cleanest and most reusable, but an extra step every time you write this query.
2. Window function, latest row only	ROW_NUMBER() OVER (PARTITION BY subscription_id ORDER BY charged_at DESC), keep only row 1.	Precise when you only care about the latest payment, but easy to misuse if you actually needed all of them summed.
3. COUNT(DISTINCT s.id) as a patch	Keep the join as-is, just dedupe the count.	Fixes the subscription count fast — but SUM(p.amount) is still wrong, since it doesn't dedupe amounts. Fixes the symptom you noticed, not the one you didn't.

Solace ships with Option 1 this level — it's the only one that fixes both numbers at once.

Fan-out fiasco

Same query. This time, pick the exact line responsible for the fan-out.

```
SELECT COUNT(s.id) AS active_subs, SUM(p.amount) AS revenue # line A
FROM subscriptions s                                     # line B
JOIN payments p ON p.subscription_id = s.id              # line C
WHERE s.status = 'active'                                # line D
```

Which line is responsible for the fan-out?

- ☐ Line A — the SELECT ☐ Line C — the JOIN ☐ Line D — the WHERE

Pick your line — the answer's on the next page.



LINE IDENTIFIED

It's Line C — the JOIN

The SELECT and WHERE just describe what you want back — the JOIN is what decides how many rows you're working with in the first place. Once payments fans out subscriptions, every aggregate downstream inherits the duplication.

Revenue is up. Something feels off.

CONFIDENTIAL

This month's revenue query says **\$184,200** and **1,540** active subscriptions. Last month: \$149,800 and 1,190. That's a bigger jump than any signup number this quarter would explain.

*After pre-aggregating payments first: **\$151,000** revenue, **1,200** active subscriptions — much closer to last month's trend.*

Before you move on

Where in your own work do you join two tables without checking if the relationship is 1-to-1 or 1-to-many?

Why does pre-aggregating fix both the count and the sum, when COUNT(DISTINCT) only fixes one?

What's one quick sanity check you could run right after any JOIN, before trusting the result?

From someone who's seen this before

Before trusting any number after a JOIN, run COUNT(*) before and after adding the join. If the row count jumped and you didn't expect it to, you have a fan-out — full stop.

The worst version of this I ever saw: a fan-out bug sat in a revenue query for four months before anyone caught it. Every board deck for a full quarter overstated MRR by about 12%. Nobody lied, nobody even made a mistake you could point to in one line — it was just a JOIN, doing exactly what JOINS do. That's the part that stays with me: the bug wasn't a bug. It was correct SQL asking the wrong question.

"1-to-many" isn't a footnote in the data dictionary — it's the single most important fact about any table you're about to join to.

The Duplicate Revenue Crisis

The board deck is due Friday, and Farrah wants to know which number goes in it — the one everyone's seen for two days, or the one you just found, three days before the meeting.

☐ **A — Quietly swap in the correct number, no explanation.**

Trade-off: fast, but if anyone compares to the number they saw Monday, it looks like something's being hidden.

☐ **B — Tell Farrah exactly what changed and why, before she has to ask.**

Trade-off: an uncomfortable conversation today, but it's the only option that builds trust instead of spending it.

☐ **C — Present both numbers and let the room decide.**

Trade-off: transparent, but hands a technical problem to people who don't have the context to resolve it.

B. Same principle every level so far: the version of you that gets trusted with bigger problems is the one who surfaces the bad news before it's found for you.

SQL Cheat Cards — Set 2 (JOINS)

INNER vs. LEFT JOIN

INNER JOIN drops rows with no match on either side. LEFT JOIN keeps every row from the left table, matched or not — know which one you actually want.

What Is Fan-Out?

Joining to a table where the relationship is 1-to-many multiplies your "one" side by however many matches exist on the "many" side.

Pre-Aggregate First

When the right-hand table is 1-to-many, summarize it into one row per key in a CTE before joining — not after.

Sanity-Check Every JOIN

Compare COUNT(*) before and after adding a join. Any unexpected increase means you just fanned something out.

Set 2 of 2 SQL cheat cards — keep both next to your desk.

Level 3, condensed

The bug: joining subscriptions to payments (1-to-many) duplicated subscriptions with multiple payment rows.

The fix: pre-aggregate payments into one row per subscription before joining.

The real numbers: \$184,200 / 1,540 subs (naive) vs. \$151,000 / 1,200 subs (correct).

The Boss Fight call: tell Farrah what changed before she has to ask.

Test Yourself on Solace's Data

Three questions, one real answer each. No skipping ahead, no peeking at the next page early. Pick your answers below, then turn the page to see how you did.

1. What made payments a dangerous table to JOIN into naively?

- ☐ A. It was missing a primary key
- ☐ B. It has multiple rows per subscription
- ☐ C. It updates in real time

2. Why doesn't COUNT(DISTINCT s.id) fully fix the revenue number?

- ☐ A. It's slower to run
- ☐ B. It only dedupes the count, not the summed amounts
- ☐ C. DISTINCT doesn't work with JOINS

3. What's the fastest sanity check right after writing any JOIN?

- ☐ A. Compare row counts before and after the join
- ☐ B. Check the query runtime
- ☐ C. Re-read the SELECT clause

Your verdict — and the answer key — is one page away.

CASE FILE VERIFIED

How'd you do?

- 1 → B.** Payments has multiple rows per subscription this month (retries, refunds) — a classic 1-to-many fan-out risk.
- 2 → B.** DISTINCT dedupes which subscriptions get counted, but SUM(p.amount) still adds every duplicated payment row.
- 3 → A.** If COUNT(*) jumps right after adding a join, you've fanned something out — catch it before you trust anything downstream.

Got all 3?

You're already thinking like the person who catches a fan-out before it hits a board deck. That instinct just saved a quarter of MRR reporting.

Got 2 out of 3?

Solid — you're tracing the JOIN, not just the SELECT. Flip back to Page 4 and Page 7 for the one that slipped past.

Got 1 or fewer?

No penalty for that in Level 3: The JOIN Labyrinth — fan-outs are genuinely sneaky. Re-read the Cheatsheet on Page 13 and carry it into Level 4.

One True Customer Table

No answer key here — map out a "one row per customer" table the way you'd actually build it, using your own work's data (or Solace's, if you'd rather practice on this case).

Your Case

Table you'd start from	What makes it 1-to-many	How you'd pre-aggregate it	What you'd sanity-check after
CLUE #1 — Which table would you start from, and why is it risky to join into directly?			
CLUE #2 — How would you pre-aggregate it into one row per key?			
CLUE #3 — What sanity check would you run right after the join?			

A third real artifact for your Level 8 portfolio project — this one shows you can design around a fan-out, not just spot one.

LEVEL COMPLETE



World 3: The JOIN Labyrinth

You finished Level 3 and caught a fan-out before it made the board deck.

Farrah asked you to review every revenue query before it ships from now on. That's not a title change — but it's the closest thing to one you've gotten so far.

"The bug wasn't a bug. It was correct SQL asking the wrong question."

NEXT UP

Level 4: The dbt Factory

You've fixed the same categories of bug three times by hand. Next, you learn to make the fix permanent — so nobody has to catch this one again.